GenericServlet class

GenericServlet class implements **Servlet**, **ServletConfig** and **Serializable** interfaces. It provides the implementation of all the methods of these interfaces except the service method.

GenericServlet class can handle any type of request so it is protocol-independent.

You may create a generic servlet by inheriting the GenericServlet class and providing the implementation of the service method.

Methods of GenericServlet class

There are many methods in GenericServlet class. They are as follows:

1. **public void init(ServletConfig config)** is used to initialize the servlet.
2. **public abstract void service(ServletRequest request, ServletResponse response)** provides service for the incoming request. It is invoked at each time when user requests for a servlet.
3. **public void destroy()** is invoked only once throughout the life cycle and indicates that servlet is being destroyed.
4. **public ServletConfig getServletConfig()** returns the object of ServletConfig.
5. **public String getServletInfo()** returns information about servlet such as writer, copyright, version etc.
6. **public void init()** it is a convenient method for the servlet programmers, now there is no need to call super.init(config)
7. **public ServletContext getServletContext()** returns the object of ServletContext.
8. **public String getInitParameter(String name)** returns the parameter value for the given parameter name.
9. **public Enumeration getInitParameterNames()** returns all the parameters defined in the web.xml file.
10. **public String getServletName()** returns the name of the servlet object.
11. **public void log(String msg)** writes the given message in the servlet log file.
12. **public void log(String msg,Throwable t)** writes the explanatory message in the servlet log file and a stack trace.

Servlet Example by inheriting the GenericServlet class

```
1. import java.io.*;
2. import javax.servlet.*;
```

```
3.
4. public class First extends GenericServlet{
5. public void service(ServletRequest req,ServletResponse res)
6. throws IOException,ServletException{
7.
8. res.setContentType("text/html");
9.
10. PrintWriter out=res.getWriter();
11. out.print("<html><body>");
12. out.print("<b>hello generic servlet</b>");
13. out.print("</body></html>");
14.
15. }
16. }
```

HttpServlet class

1. HttpServlet class
2. Methods of HttpServlet class

The HttpServlet class extends the GenericServlet class and implements Serializable interface. It prov
specific methods such as doGet, doPost, doHead, doTrace etc.

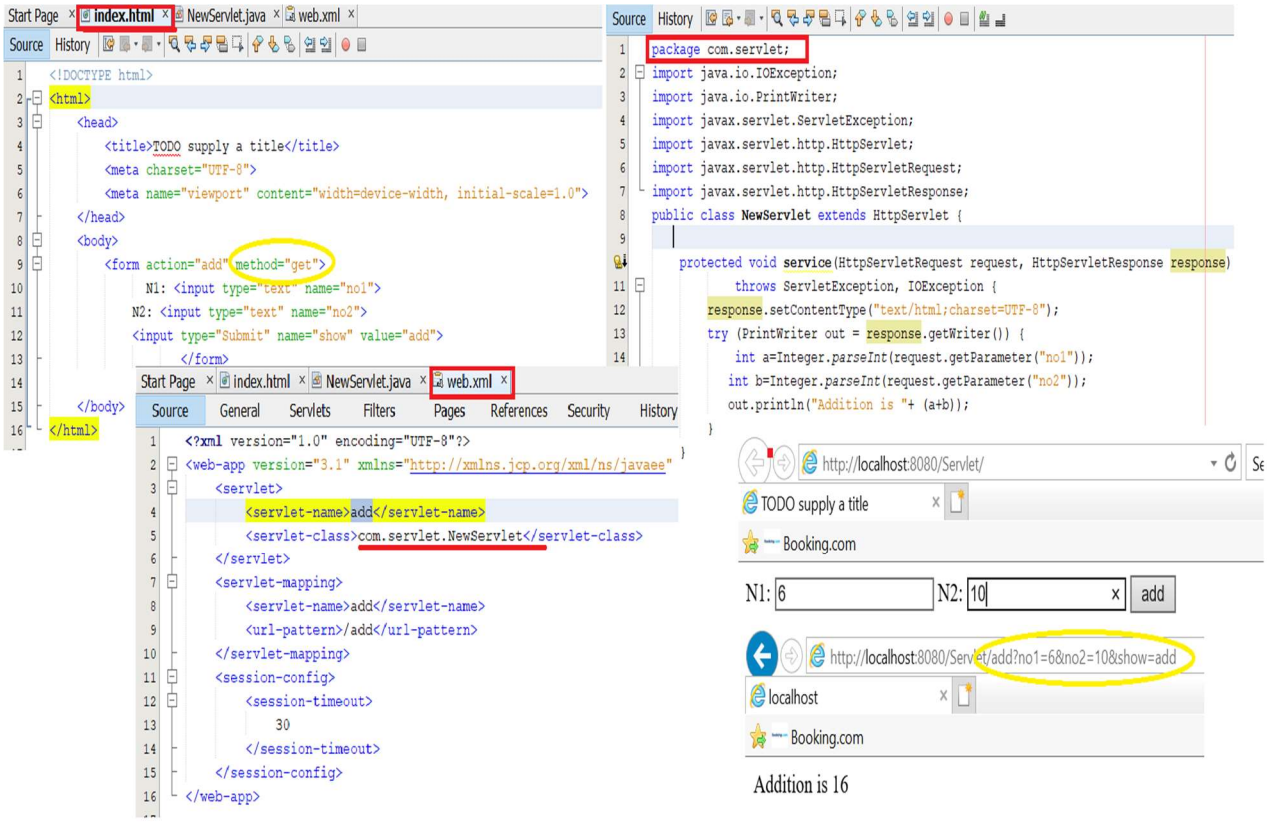
Methods of HttpServlet class

There are many methods in HttpServlet class. They are as follows:

1. **public void service(ServletRequest req,ServletResponse res)** dispatches the request to the protected service method by converting the request and response object into http type.
2. **protected void service(HttpServletRequest req, HttpServletResponse res)** receives the request from the service method, and dispatches the request to the doXXX() method depending on the incoming http request type.
3. **protected void doGet(HttpServletRequest req, HttpServletResponse res)** handles the GET request. It is invoked by the web container.
4. **protected void doPost(HttpServletRequest req, HttpServletResponse res)** handles the POST request. It is invoked by the web container.
5. **protected void doHead(HttpServletRequest req, HttpServletResponse res)** handles the HEAD request. It is invoked by the web container.
6. **protected void doOptions(HttpServletRequest req, HttpServletResponse res)** handles the OPTIONS request. It is invoked by the web container.

7. **protected void doPut(HttpServletRequest req, HttpServletResponse res)** handles the PUT request. It is invoked by the web container.
8. **protected void doTrace(HttpServletRequest req, HttpServletResponse res)** handles the TRACE request. It is invoked by the web container.
9. **protected void doDelete(HttpServletRequest req, HttpServletResponse res)** handles the DELETE request. It is invoked by the web container.
10. **protected long getLastModified(HttpServletRequest req)** returns the time when HttpServletRequest was last modified since midnight January 1, 1970 GMT.

Simple Example: Http Servlet (Addition of two Numbers)



➢ Handling HTTP get Requests, Handling HTTP post Requests

Difference Between dopost and doget in Servlet

Http protocol mostly use either get or post methods to transfer the request. post method are generally used whenever you want to transfer secure data like password, bank account etc.

	Get method	Post method
1	Get Request sends the request parameter as query string appended at the end of the request.	Post request send the request parameters as part of the http request body.
2	Get method is visible to every one (It will be displayed in the address bar of browser).	Post method variables are not displayed in the URL.
3	Restriction on form data, only ASCII characters allowed.	No Restriction on form data, Binary data is also allowed.
4	Get methods have maximum size is 2000 character.	Post methods have maximum size is 8 mb.
5	Restriction on form length, So URL length is restricted	No restriction on form data.
6	Remain in browser history.	Never remain the browser history.

GET Method:

Start Page | index1.html | NewServlet1.java

Source | History

```java
package com.servlet;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/NewServlet1")
public class NewServlet1 extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)throws ServletException, IOException {

        res.getWriter().append("Welcome").append(req.getParameter("fname"));

    }
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)throws ServletException, IOException {
        // response.setContentType("text/html;charset=UTF-8");
        // res.getWriter().append("Welcome").append(req.getParameter("fname"));


    }
}
```

Start Page | index1.html | NewServlet1.java

Source | History | C:\Users\deept\OneDrive\Documents\NetBeansProjects\Servlet\web\inc

```html
<!DOCTYPE html>
<html>
    <head>
        <title>TODO supply a title</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initi
    </head>
    <body>
        <form name="myform" method="get" action=NewServlet1>
            Name: <input type="text" name="fname">

            <input type="Submit" value="ok">
                </form>

    </body>
</html>
```

http://localhost:8080/Servlet/NewServlet1?fname=Deepti

localhost

Booking.com

WelcomeDeepti

Post Method:

index1.html | NewServlet1.java

Source | History

```html
<!DOCTYPE html>
<html>
    <head>
        <title>TODO supply a title</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1
    </head>
    <body>
        <form name="myform" method="post" action=NewServlet1>
            Name: <input type="text" name="fname">

            <input type="Submit" value="ok">
                </form>

    </body>
</html>
```

http://localhost:8080/Servlet/index1.html

TODO supply a title

Booking.com

Name: Deepti ok

http://localhost:8080/Servlet/NewServlet1

localhost

Booking.com

WelcomeDeepti

index1.html | NewServlet1.java

Source | History

```java
package com.servlet;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/NewServlet1")
public class NewServlet1 extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)throws ServletException, IOException

        //res.getWriter().append("Welcome").append(req.getParameter("fname"));

    }
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)throws ServletException, IOException {
        // response.setContentType("text/html;charset=UTF-8");
        res.getWriter().append("Welcome").append(req.getParameter("fname"));


    }
}
```

SendRedirect in servlet

The **sendRedirect()** method of **HttpServletResponse** interface can be used to redirect response to another resource, it may be servlet, jsp or html file.
It accepts relative as well as absolute URL.
It works at client side because it uses the url bar of the browser to make another request. So, it can work inside and outside the server.

Difference between forward() and sendRedirect() method

There are many differences between the forward() method of RequestDispatcher and sendRedirect() method of HttpServletResponse interface. They are given below:

forward() method	sendRedirect() method
The forward() method works at server side.	The sendRedirect() method works at client side.
It sends the same request and response objects to another servlet.	It always sends a new request.
It can work within the server only.	It can be used within and outside the server.
Example: request.getRequestDispacher("servlet2").forward (request,response);	Example: response.sendRedirect("servlet2");

Syntax of sendRedirect() method

```
public void sendRedirect(String URL)throws IOException;
```

Example of sendRedirect() method

```
response.sendRedirect("http://www.....com");
```

index3.html × NewServlet2.java ×

Source History

```html
<!DOCTYPE html>
<html>
    <head>
        <title>Redirect page</title>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
    </head>
    <body>
        <form action=NewServlet2>
            Name: <input type="text" name="name">

        <input type="Submit" value="search">
            </form>
    </body>
</html>
```

1

index3.html × NewServlet2.java ×

Source History

```java
package com.servlet;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/NewServlet2")
public class NewServlet2 extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String name=request.getParameter("name");
        response.sendRedirect("https://www.google.com/#q=" + name);

        }
    }
```

2

http://localhost:8080/Servlet/index3.html

Redirect page

Booking.com

Name: facebook search

3

4

Google facebook

All Books Shopping News Images More Settings Tools

About 24,79,00,000 results (0.40 seconds)

Ad · www.facebook.com/Sign_Up

Facebook® Account Sign Up | Facebook.com

Connect With Your Friends Online. Join The Facebook Community - Free! Share Photos And Videos, Messages And Get Updates From Your Communities. Chat Online. Connect With Old Friends. Plan Events. Play Games With Friends. Stay In Touch With Family. Find Friends.

Sign Up For Facebook®
Stay Connected with People Around The World - Anywhere, Anytime.

Facebook® for Mobile
Stay Connected On The Go, Anywhere, Anytime. Download The App Today!

Search On Facebook®
Find The People and Posts That Matter To You Most. Try A Search!

Facebook® Photo Stream
Find Photos and Videos On Facebook, Or Upload Your Own Today.

www.facebook.com

Session Tracking in Servlets

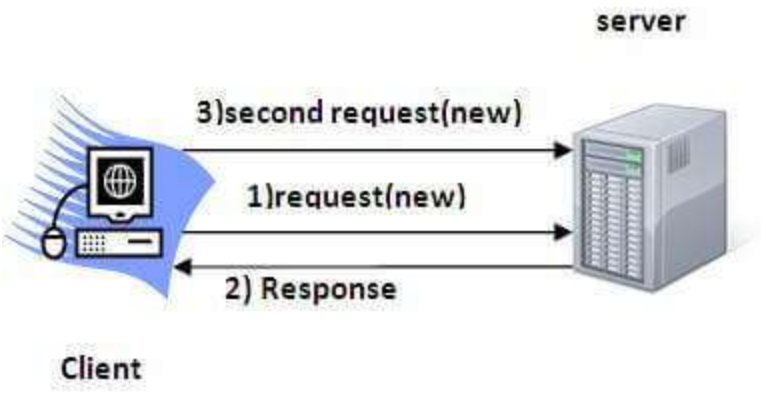
1. Session Tracking
2. Session Tracking Techniques

Session simply means a particular interval of time.

Session Tracking is a way to maintain state (data) of an user. It is also known as **session management** in servlet.

Http protocol is a stateless so we need to maintain state using session tracking techniques. Each time user requests to the server, server treats the request as the new request. So we need to maintain the state of an user to recognize to particular user.

HTTP is stateless that means each request is considered as the new request. It is shown in the figure given below:



Why use Session Tracking?

To recognize the user It is used to recognize the particular user.

Session Tracking Techniques

There are four techniques used in Session tracking:

1. **Cookies**
2. **Hidden Form Field**
3. **URL Rewriting**
4. **HttpSession**

Cookies in Servlet

A **cookie** is a small piece of information that is persisted between the multiple client requests.

A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.

How Cookie works

By default, each request is considered as a new request. In cookies technique, we add cookie with response from the servlet. So cookie is stored in the cache of the browser. After that if request is sent by the user, cookie is added with request by default. Thus, we recognize the user as the old user.

Types of Cookie

There are 2 types of cookies in servlets.

1. Non-persistent cookie
2. Persistent cookie

Non-persistent cookie

It is **valid for single session** only. It is removed each time when user closes the browser.

Persistent cookie

It is **valid for multiple session** . It is not removed each time when user closes the browser. It is removed only if user logout or signout.

Advantage of Cookies

1. Simplest technique of maintaining the state.
2. Cookies are maintained at client side.

Disadvantage of Cookies

1. It will not work if cookie is disabled from the browser.
2. Only textual information can be set in Cookie object.

Cookie class

javax.servlet.http.Cookie class provides the functionality of using cookies. It provides a lot of useful methods for cookies.

Constructor of Cookie class

Constructor	Description
Cookie()	constructs a cookie.
Cookie(String name, String value)	constructs a cookie with a specified name and v

Useful Methods of Cookie class

There are given some commonly used methods of the Cookie class.

Method	Description
public void setMaxAge(int expiry)	Sets the maximum age of the cookie in seconds.
public String getName()	Returns the name of the cookie. The name cannot be char
public String getValue()	Returns the value of the cookie.
public void setName(String name)	changes the name of the cookie.
public void setValue(String value)	changes the value of the cookie.

Other methods required for using Cookies

For adding cookie or getting the value from the cookie, we need some methods provided by other in

1. **public void addCookie(Cookie ck):**method of HttpServletResponse interface is used to add object.
2. **public Cookie[] getCookies():**method of HttpServletRequest interface is used to return all browser.

How to create Cookie?

Let's see the simple code to create cookie.

```
1. Cookie ck=new Cookie("user","sonoo jaiswal");//creating cookie object
2. response.addCookie(ck);//adding cookie in the response
```

How to delete Cookie?

Let's see the simple code to delete cookie. It is mainly used to logout or signout the user.

```
1. Cookie ck=new Cookie("user","");//deleting value of cookie
2. ck.setMaxAge(0);//changing the maximum age to 0 seconds
3. response.addCookie(ck);//adding cookie in the response
```

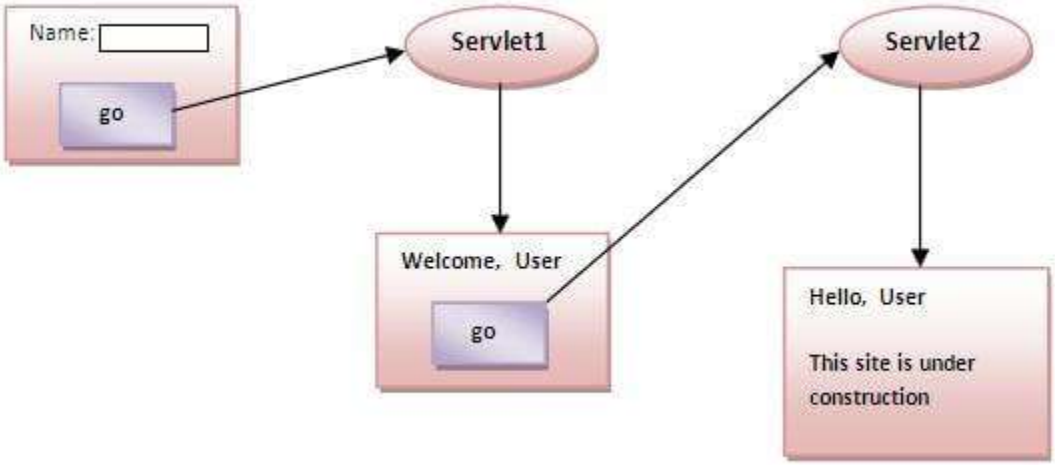
How to get Cookies?

Let's see the simple code to get all the cookies.

```
1. Cookie ck[]=request.getCookies();
2. for(int i=0;i<ck.length;i++){
3.  out.print("<br>"+ck[i].getName()+" "+ck[i].getValue());//printing name and value of c
   ookie
4. }
```

Simple example of Servlet Cookies

In this example, we are storing the name of the user in the cookie object and accessing it in another servlet. As we know well that session corresponds to the particular user. So if you access it from too many browsers with different values, you will get the different value.



index.html

```
1. <form action="servlet1" method="post">
2. Name:<input type="text" name="userName"/><br/>
3. <input type="submit" value="go"/>
4. </form>
```

FirstServlet.java

```
1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5.
6. public class FirstServlet extends HttpServlet {
7.
8.   public void doPost(HttpServletRequest request, HttpServletResponse response){
9.     try{
10.
11.    response.setContentType("text/html");
12.    PrintWriter out = response.getWriter();
13.
14.    String n=request.getParameter("userName");
15.    out.print("Welcome "+n);
16.
17.    Cookie ck=new Cookie("uname",n);//creating cookie object
18.    response.addCookie(ck);//adding cookie in the response
19.
20.    //creating submit button
```

```
21.     out.print("<form action='servlet2'>");
22.     out.print("<input type='submit' value='go'>");
23.     out.print("</form>");
24.
25.     out.close();
26.
27.         }catch(Exception e){System.out.println(e);}
28.   }
29. }
```

SecondServlet.java

```
1.  import java.io.*;
2.  import javax.servlet.*;
3.  import javax.servlet.http.*;
4.
5.  public class SecondServlet extends HttpServlet {
6.
7.  public void doPost(HttpServletRequest request, HttpServletResponse response){
8.      try{
9.
10.     response.setContentType("text/html");
11.     PrintWriter out = response.getWriter();
12.
13.     Cookie ck[]=request.getCookies();
14.     out.print("Hello "+ck[0].getValue());
15.
16.     out.close();
17.
18.         }catch(Exception e){System.out.println(e);}
19.     }
20.
21.
22. }
```

web.xml

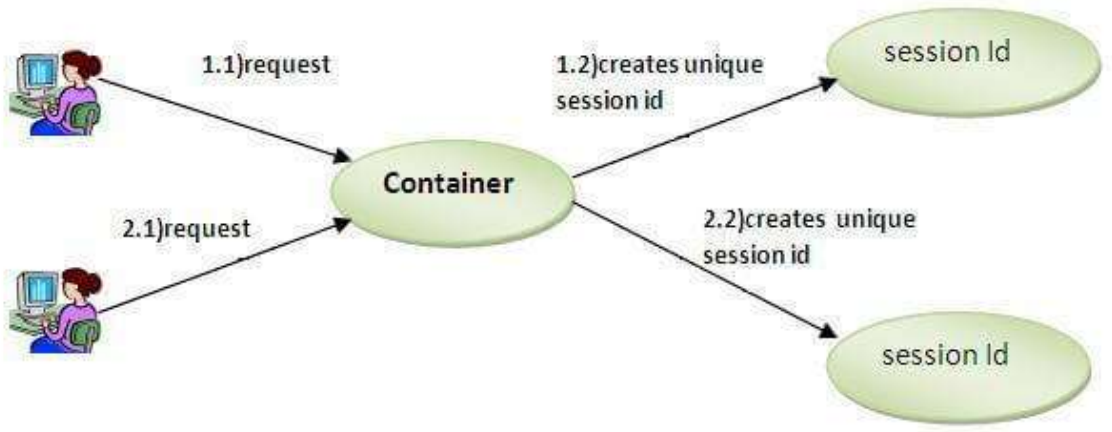
```
1.  <web-app>
2.
3.  <servlet>
4.  <servlet-name>s1</servlet-name>
5.  <servlet-class>FirstServlet</servlet-class>
6.  </servlet>
7.
```

```
8.  <servlet-mapping>
9.  <servlet-name>s1</servlet-name>
10. <url-pattern>/servlet1</url-pattern>
11. </servlet-mapping>
12.
13. <servlet>
14. <servlet-name>s2</servlet-name>
15. <servlet-class>SecondServlet</servlet-class>
16. </servlet>
17.
18. <servlet-mapping>
19. <servlet-name>s2</servlet-name>
20. <url-pattern>/servlet2</url-pattern>
21. </servlet-mapping>
22.
23. </web-app>
```

HttpSession interface

In such case, container creates a session id for each user.The container uses this id to identify the particular user.An object of HttpSession can be used to perform two tasks:

1. bind objects
2. view and manipulate information about a session, such as the session identifier, creation time, and last accessed time.



How to get the HttpSession object ?

The HttpServletRequest interface provides two methods to get the object of HttpSession:

1. **public HttpSession getSession():**Returns the current session associated with this request, or if the request does not have a session, creates one.
2. **public HttpSession getSession(boolean create):**Returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session.

Commonly used methods of HttpSession interface

1. **public String getId():**Returns a string containing the unique identifier value.
2. **public long getCreationTime():**Returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.
3. **public long getLastAccessedTime():**Returns the last time the client sent a request associated with this session, as the number of milliseconds since midnight January 1, 1970 GMT.
4. **public void invalidate():**Invalidates this session then unbinds any objects bound to it.

Example of using HttpSession

In this example, we are setting the attribute in the session scope in one servlet and getting that value from the session scope in another servlet. To set the attribute in the session scope, we have used the setAttribute() method of HttpSession interface and to get the attribute, we have used the getAttribute method.

index.html

```
1. <form action="servlet1">
2. Name:<input type="text" name="userName"/><br/>
3. <input type="submit" value="go"/>
4. </form>
```

FirstServlet.java

```
1. import java.io.*;
2. import javax.servlet.*;
3. import javax.servlet.http.*;
4.
5.
6. public class FirstServlet extends HttpServlet {
7.
8. public void doGet(HttpServletRequest request, HttpServletResponse response){
9.         try{
10.
```

```
11.         response.setContentType("text/html");
12.         PrintWriter out = response.getWriter();
13.
14.         String n=request.getParameter("userName");
15.         out.print("Welcome "+n);
16.
17.         HttpSession session=request.getSession();
18.         session.setAttribute("uname",n);
19.
20.         out.print("<a href='servlet2'>visit</a>");
21.
22.         out.close();
23.
24.                 }catch(Exception e){System.out.println(e);}
25.     }
26.
27. }
```

SecondServlet.java

```
1.  import java.io.*;
2.  import javax.servlet.*;
3.  import javax.servlet.http.*;
4.
5.  public class SecondServlet extends HttpServlet {
6.
7.  public void doGet(HttpServletRequest request, HttpServletResponse response)
8.          try{
9.
10.         response.setContentType("text/html");
11.         PrintWriter out = response.getWriter();
12.
13.         HttpSession session=request.getSession(false);
14.         String n=(String)session.getAttribute("uname");
15.         out.print("Hello "+n);
16.
17.         out.close();
18.
19.                 }catch(Exception e){System.out.println(e);}
20.     }
21.
22.
```

```
23. }
```

web.xml

```
1.  <web-app>
2.
3.  <servlet>
4.  <servlet-name>s1</servlet-name>
5.  <servlet-class>FirstServlet</servlet-class>
6.  </servlet>
7.
8.  <servlet-mapping>
9.  <servlet-name>s1</servlet-name>
10. <url-pattern>/servlet1</url-pattern>
11. </servlet-mapping>
12.
13. <servlet>
14. <servlet-name>s2</servlet-name>
15. <servlet-class>SecondServlet</servlet-class>
16. </servlet>
17.
18. <servlet-mapping>
19. <servlet-name>s2</servlet-name>
20. <url-pattern>/servlet2</url-pattern>
21. </servlet-mapping>
22.
    </web-app>
```